Nodra DocType: Complete Guide and API Reference

This guide covers every part of a DocType in `nodra-workspace` (`packages/nodra`): the JSON definition, fields, naming, relations, controllers, validation, permissions, migrations, and the full API (Registry, Document, ORM, errors, CLI). It is written from the source code, so it also tells you what is **enforced** today and what is **stored only** (accepted in the JSON and saved as metadata, but not acted on by the ORM).

Legend: *Enforced* means the framework acts on it. *Stored only* means it is parsed and saved to the metadata tables, but nothing in the ORM, validator or schema sync uses it yet.

Contents: 1 Concepts, 2 Quick start, 3 DocType properties, 4 Fields, 5 Standard fields, 6 Naming, 7 Relations, 8 Controllers and hooks, 9 Validation, 10 Permissions, 11 Submittable and workflow, 12 Migrations, 13 API reference, 14 Gotchas, 15 Troubleshooting, 16 Cheat sheet.

---

1. Concepts

Term	Meaning
DocType	A JSON file that describes one kind of record (its fields, naming, permissions)
Document	One record of a DocType, as a TypeScript object (`Document` class)
Controller	A class that extends `Document` and overrides lifecycle hooks
Registry	`app.registry` , the in-memory list of all loaded DocTypes
ORM	`app.orm` , which saves and loads Documents (insert, getDoc, getList, update, deleteDoc)
Migrate	The CLI command that creates or extends database tables from your JSON

How a DocType becomes a table and a record

1. You write `doctypes/<name>/<name>.json`.
2. At startup the **loader** reads it, validates it with `parseDocType`, and adds the standard fields (`injectStandardFields`).
3. The definition is stored in the **registry** (in memory).
4. `pnpm run migrate` creates the table `tab_<snake_case_name>` and saves metadata rows.
5. The **ORM** reads and writes rows, wrapping each one in a `Document`.

File layout and loading rules

```
apps/<app>/doctypes/
  leave-request/
    leave-request.json     <- the definition
    leave-request.ts       <- optional controller
```

- The loader scans the folder **recursively** and treats **every `.json` file** as a DocType. A file that fails validation is skipped with a console warning (`Skipping <path>: <reason>`). If a DocType seems to be missing, check the server log first.
- At app boot, framework DocTypes (`packages/nodra/doctypes`, such as User and Role) load first, then your app's `doctypes/` folder, so your DocTypes can link to core ones.
- Registering two DocTypes with the same `name` throws `DuplicateError`.
- DocTypes are read at boot. After changing a JSON file, run migrate (if columns changed) and restart the server.

---

2. Quick start

```
pnpm run new:doctype -- "Leave Request"   # 1. scaffold JSON + controller
# 2. edit leave-request.json
pnpm run migrate                          # 3. create or extend the table
# 4. restart the dev server
```

A minimal valid DocType:

```
{
  "name": "Leave Request",
  "module": "HR",
  "naming_rule": "hash",
  "is_submittable": false,
  "is_child": false,
  "is_single": false,
  "is_tree": false,
  "is_virtual": false,
  "fields": [
    { "fieldname": "employee_name", "fieldtype": "Data", "label": "Employee",
"reqd": true }
  ],
  "permissions": [
    { "role": "System Manager", "read": true, "write": true, "create": true,
"delete": true }
  ]
}
```

Then use it:

```
import { Document } from 'nodra/core/document/document';
import { getNodra } from '../nodra-app';

const app = await getNodra();
const doc = new Document(app.registry.get('Leave Request'), { employee_name:
'Ravi' });
await app.orm.insert(doc);                          // doc.name is now set
const again = await app.orm.getDoc('Leave Request', doc.name);
```

---

3. DocType properties

Key	Type	Required	Default	Meaning	Status
`name`	string	yes	none	Unique name. Table name is `tab_` plus snake_case (`Sales Order` becomes `tab_sales_order`)	Enforced

Key	Type	Required	Default	Meaning	Status
`module`	string	yes	none	Group or app area the DocType belongs to	Metadata, used by `listByModule`
`fields`	array	yes, at least 1	none	The field definitions (section 4)	Enforced
`naming_rule`	string	no	`autoincrement`	One of `autoincrement`, `hash`, `field`, `format`, `prompt`, `expression`	Parsed. The ORM currently generates a hash for all of them (section 6)
`is_child`	boolean	no	false	Marks a child table. Adds `parent`, `parenttype`, `parentfield` columns and indexes	Enforced
`permissions`	array	no	empty	Role rules (section 10)	Enforced by `hasPermission` (not by the ORM)
`field_permissions`	array	no	none	Per-field read/write rules	Enforced by the field-permission helpers
`is_submittable`	boolean	no	false	Intended for submit/cancel flow	Stored only
`is_single`	boolean	no	false	Intended for one global record	Stored only
`is_tree`	boolean	no	false	Intended for nested records	Stored only
`is_virtual`	boolean	no	false	Intended for DocTypes with no table	Stored only (a table is still created)
`title_field`	string	no	none	Field used as display title	Stored only
`search_fields`	string[]	no	none	Fields for search	Stored only
`sort_field`, `sort_order`	string, `asc` / `desc`	no	none	Default list sort	Stored only (`sort_field` is saved to metadata)
`hooks`	object	no	none	Declarative hooks	Stored only

Notes

- `naming_field` appears in the core User JSON, but `parseDocType` does not keep it, so it has no effect.
- Boolean keys are coerced with `Boolean(...)`, so omit them or use real `true` / `false`.
- `permissions` entries default each missing action to `false`.

Parse errors you may see (all `ValidationError`): `"name" is required...`, `"module" is required...`, `"fields" is required and must be an array`, `"fields" must contain at least one field`, `invalid naming_rule "x"`, `invalid fieldtype "x"`, and `"label" is required...`.

---

4. Fields

4.1 Field properties

Every field needs `fieldname`, `fieldtype` and `label`. Everything else is optional.

Property	Type	Effect	Status
`fieldname`	string	Column name. Use snake_case	Enforced
`fieldtype`	string	Column type and validation (section 4.2)	Enforced
`label`	string	Display label	Metadata
`reqd`	boolean	`NOT NULL` in the database, and the validator rejects null, undefined or empty string	Enforced
`unique`	boolean	`UNIQUE` constraint in the database	Enforced (by the database)

Property	Type	Effect	Status
`default`	any	`DEFAULT` value in the database	Enforced (by the database only, see below)
`max_length`	number	`VARCHAR(n)` for `Data`; the validator rejects longer strings for any field	Enforced
`options`	string or string[]	`Select`: allowed values. `Link` / `Table`: target DocType	Enforced (`Select` only when it is an array)
`search_index`	boolean	Creates a database index	Enforced
`hidden`, `read_only`, `in_list_view`, `in_standard_filter`, `description`, `depends_on`	various	UI hints	Metadata only. `read_only` is not enforced by the ORM
`precision`	number	Decimal precision hint	Stored only (`Currency` is always `NUMERIC(18,6)`)

Defaults need care. `default` becomes a database `DEFAULT`. The ORM does not apply defaults in code. It omits fields whose value is `undefined`, so the database fills them in. But the validator runs **before** the insert and sees `undefined`, so a field that is both `reqd` and has a `default` still fails with *is a mandatory field*. Either set the value in code or do not mark it `reqd`.

`max_length` only changes the column for `Data` (`VARCHAR(max_length)`). A `Data` field without it is `VARCHAR(255)`, and the validator does not stop a longer string, so the database throws instead.

Field name tips. Avoid reserved words (`user`, `order`, `group`, `check`, `default`, `index` and so on). The DDL quotes them but the ORM's SQL does not, so queries are likely to fail.

4.2 Field types

Type	PostgreSQL column	Validator checks	Notes
`Data`	VARCHAR(255) or VARCHAR(max_length)	max_length	Short text
`Text`, `SmallText`, `LongText`	TEXT	none	All three are the same column
`Int`	INTEGER	must be a whole JS number	The string `'5'` fails
`Float`	DOUBLE PRECISION	must be a number	
`Currency`	NUMERIC(18,6)	must be a number	Values may be read back as strings, so wrap with `Number()`
`Date`, `Datetime`, `Time`	DATE, TIMESTAMPTZ, TIME	a valid `Date` or a parseable string	
`Check`	BOOLEAN	boolean or 0/1	
`Select`	VARCHAR(255)	value must be in `options` (when it is an array)	
`Link`	VARCHAR(255) plus index	none	Stores the other record's `name`. No existence check
`DynamicLink`	VARCHAR(255) plus index	none	Link whose target DocType is chosen at runtime
`Table`	no column	none	Child rows live in another table (section 7)

Type	PostgreSQL column	Validator checks	Notes
`Attach`, `AttachImage`	TEXT	none	Store a path or URL. No upload handling in the ORM
`Color`	VARCHAR(7)	none	e.g. `#ff8800`
`JSON`	JSONB	none	See the tip below
`Password`	TEXT	none	**Not hashed** by the ORM. Hash it yourself
`ReadOnly`	no column	none	Virtual field
`HTML`	TEXT	none	

JSON tip. The `pg` driver turns JavaScript arrays into PostgreSQL arrays, which JSONB rejects. If you store an array in a `JSON` field, pass `JSON.stringify(value)`. Plain objects work as they are.

4.3 Select example

```
{ "fieldname": "status", "fieldtype": "Select", "label": "Status",
  "options": ["Draft", "Confirmed", "Cancelled"], "default": "Draft" }
```

`options` must be a JSON **array** for the validator to check it. A newline-separated string is accepted by the parser, but then no check is made.

---

5. Standard fields

These are added to **every** DocType. Never declare them yourself.

Field	Type	Meaning
`name`	Data	Primary identifier of the record
`owner`	Data	Creator (defaults to `Administrator`)
`creation`	Datetime	Set on insert

Field	Type	Meaning
`modified`	Datetime	Set on every insert and update
`modified_by`	Data	Last user (defaults to owner)
`docstatus`	Int	0 Draft, 1 Submitted, 2 Cancelled. Saved by `insert` and `update`
`idx`	Int	Row order, used for child rows

Child DocTypes (`is_child: true`) also get `parent` , `parenttype` and `parentfield` .

Important: as generated today, the `name` column has no `PRIMARY KEY` or `UNIQUE` constraint, so the database will not stop two rows with the same name. When you set a custom `name` , check with `orm.exists` first.

On a Document these seven are real properties (`doc.name` , `doc.docstatus` …). Everything else goes through `doc.get()` and `doc.set()` .

---

6. Naming

`naming_rule` says how a record gets its `name` . The schema accepts six values, but today `ORM.insert()` only generates a **random 10-character hex hash**, whatever the rule says. The name is generated only when `doc.name` is empty.

Rule	Intended meaning	What happens today
`hash`	Random name	Works: 10 hex characters
`autoincrement`	Numeric sequence	Falls back to hash
`field`	Name from a field's value	Falls back to hash. Set `name` yourself
`format`	Pattern such as `LR-{#####}`	Falls back to hash. Set `name` yourself
`prompt`	Supplied by the user	Pass `name` in your data
`expression`	Computed	Falls back to hash. Set `name` yourself

Doing it yourself

Because an explicit `name` is respected, you can implement any rule in your server code with the helpers from `nodra/core/doctype/naming`:

```
import { generateFieldName, parseFormatPattern } from
'nodra/core/doctype/naming';

// field rule: name = the customer's name
const name = generateFieldName(input.customer_name);   // trims, rejects
empty
if (await app.orm.exists('Customer', name)) throw new Error('Customer already
exists');

// format rule: LR-00001, LR-00002, ...
const next = (await app.orm.getCount('Leave Request')) + 1;
const formatted = parseFormatPattern('LR-{#####}', next);   // 'LR-00001'
```

The counter above is **not safe** when two requests run at once (both may read the same count). For real sequences, use a database sequence or a counter row with a locked update.

---

7. Relations

7.1 Link (many to one)

```
{ "fieldname": "customer", "fieldtype": "Link", "label": "Customer",
"options": "Customer", "reqd": true }
```

- Stores the target record's `name` as text and creates an index.
- It is **not** a foreign key, and the validator does not check that the target exists.
- Check before saving: `await app.orm.exists('Customer', value)`.
- Check before deleting the target: `await app.orm.getCount('Sales Order', { customer: name })`.
- `app.registry.getLinkedDocTypes('Customer')` lists DocTypes that have a `Link` field pointing at it. It does not include `DynamicLink` fields.

7.2 Child table (one to many)

Child DocType (`is_child: true`) plus a `Table` field on the parent:

```
{ "fieldname": "items", "fieldtype": "Table", "label": "Items", "options":
"Sales Order Item" }
```

Rules:

- A `Table` field has **no column**. The ORM does not save or load child rows for you.
- Never set a value for the `Table` field on the parent Document (it would try to write a missing column), and never mark it `reqd`.
- Insert each child as its own Document with `parent`, `parenttype`, `parentfield` and `idx`.
- Read children with `getList(child, { filters: { parent: name }, orderBy: 'idx asc' })`.
- Delete children yourself before the parent. There is no cascade.

```
await app.orm.insert(new Document(app.registry.get('Sales Order Item'), {
  parent: order.name, parenttype: 'Sales Order', parentfield: 'items',
  idx: 1, item: 'PRD-1', qty: 2, rate: 100, amount: 200,
}));
```

7.3 DynamicLink

A `DynamicLink` is a text column holding a record name, where the target DocType is stored in another field of your own (for example `reference_type` and `reference_name`). The framework adds the index but does no lookup or validation. Resolve it yourself: `getDoc(doc.get('reference_type'), doc.get('reference_name'))`.

7.4 Many to many

Use a small child DocType holding a `Link` (like the core `User Role` child of `User`), or a standalone DocType with two `Link` fields.

---

8. Controllers and lifecycle hooks

A controller is a class that extends `Document`. Override only the hooks you need.

```
import { Document } from 'nodra/core/document/document';
import { ValidationError } from 'nodra/core/errors';

export class LeaveRequest extends Document {
  override async validate(): Promise<void> {
    const from = new Date(String(this.get('from_date')));
    const to = new Date(String(this.get('to_date')));
    if (to < from) throw new ValidationError('To date cannot be before From
date');
  }

  override async afterInsert(): Promise<void> {
    // e.g. send a notification
  }
}
```

Hook order

Operation	Hooks, in order
`insert`	beforeValidate, validate, built-in field checks, beforeInsert, beforeSave, **INSERT**, afterSave, afterInsert
`update`	beforeValidate, validate, built-in field checks, beforeSave, **UPDATE**, afterSave, onChange
`deleteDoc`	beforeDelete, **DELETE**, afterDelete

A hook that throws stops the operation. If it throws before the SQL runs, nothing is saved.

Change tracking inside hooks

Method	Returns
`isNew()`	true until the document is saved or loaded from the database
`hasChanged(field)`	true if the value differs from the last clean snapshot (compared with `!==`, so objects and dates compare by reference)

Method	Returns
`getPrevious(field)`	the value at the last snapshot
`getChangedFields()`	list of changed field names
`markAsClean()`	makes the current values the new baseline (the ORM calls it after a save)

Example, only act when a field changes:

```
override async beforeSave() {
  if (this.hasChanged('status') && this.get('status') === 'Confirmed') {
    this.set('confirmed_on', new Date());
  }
}
```

Three controller gotchas

1. **Controllers are not auto-discovered.** The ORM runs the hooks of the object you pass in. Build the controller yourself (`new LeaveRequest(meta, data)`); a plain `Document` runs no custom hooks.
2. **Loaded documents are plain `Document` s.** `getDoc` and `getList` return base `Document` objects. To run your hooks on an existing record, wrap it:

```
const row = await app.orm.getDoc('Leave Request', name);
const doc = new LeaveRequest(app.registry.get('Leave Request'), {
...row.getData(), _isNew: false });
doc.markAsClean();
doc.set('status', 'Approved');
await app.orm.update(doc);          // now LeaveRequest hooks run
```

3. **`deleteDoc` always loads a plain `Document` ,** so your controller's `beforeDelete` and `afterDelete` never run (this includes the built-in Role protection). Do the checks yourself before calling `deleteDoc` , or change `ORM.deleteDoc` to accept a controller.

---

9. Validation

After your `validate()` hook, the ORM calls `validateDocument(doc, meta)` for every field in the DocType. It does **not** stop at the first problem. It collects all of them and throws one `ValidationError`.

Check	Rule
Mandatory	`reqd` fields cannot be null, undefined or an empty string. `0` and `false` are valid
Type	`Int` needs a whole number, `Float` and `Currency` a number, `Check` a boolean or 0/1, date types a valid date
Length	strings longer than `max_length` are rejected
Select	the value must be in `options` (array form)

Not checked: that a `Link` target exists, `unique` (the database does that), `read_only`, `depends_on`, `Password` strength.

Handling the error:

```
import { ValidationError } from 'nodra/core/errors';

try {
  await app.orm.insert(doc);
} catch (e) {
  if (e instanceof ValidationError) {
    console.log(e.message);   // 'Validation failed for Customer: 2 error(s)'
    console.log(e.details);   // [{ field: 'customer_name', message: '...' },
...]
  }
}
```

`ValidationError` carries HTTP status 400, so server code can return it straight to the client.

---

10. Permissions

Permissions are plain functions you call from your server code. **The ORM does not check permissions itself**, so every server function that reads or writes data should call them first.

10.1 DocType permission rules

```
"permissions": [
  { "role": "System Manager", "read": true, "write": true, "create": true,
"delete": true },
  { "role": "Employee", "read": true, "create": true, "if_owner": true }
]
```

Key	Meaning
`role`	The role this rule is for
`read`, `write`, `create`, `delete`	Basic actions
`submit`, `cancel`, `amend`	Submit-flow actions (checked by `hasPermission`, but nothing in the ORM performs them)
`if_owner`	The rule applies only when the document's `owner` equals the user's email

How `hasPermission(doctype, action, user, documentOwner?)` decides:

1. A user with the role **System Manager** is always allowed.
2. Otherwise take the rules whose `role` is one of the user's roles. If there are none, the answer is **no**.
3. Skip any rule with `if_owner` unless the user is the owner of the document.
4. If any remaining rule grants the action, the answer is **yes**.

```
import { hasPermission, assertPermission } from
'nodra/permissions/permission';

const user = { email: 'ravi@shop.com', roles: ['Employee'] };
const meta = app.registry.get('Leave Request');

hasPermission(meta, 'read', user);                          // true or false
assertPermission(meta, 'write', user, doc.owner);           // throws
PermissionError (403)
```

Other helpers: `getAccessibleDocTypes(doctypes, user)` returns the names the user can read. The app template also uses `getRoleHierarchy(roles, hierarchy)` to add inherited roles.

Where roles come from. In the app template, `getUserRoles` reads `user.get('roles')`, but the ORM does not load child rows, so that value is always empty and everyone becomes `Guest`. Until it is fixed, load roles from the child table: `getList('User Role', { filters: { parent: email, parenttype: 'User' } })`.

10.2 Field permissions

```
"field_permissions": [
  { "role": "Employee", "read": ["employee_name", "status"], "write":
["reason"] },
  { "role": "HR Manager", "read": ["*"], "write": ["*"] }
]
```

- `['*']` means all fields. Only the **first** rule matching one of the user's roles is used (rules are not combined).
- `System Manager` and `Admin` bypass the check. A field marked `read_only` can never be written.
- An optional `condition` is a JavaScript expression with `doc` and `user` in scope. It is run with `new Function`, so only developers should ever write it; never build it from user input.

Function	Purpose
`hasFieldPermission(doctype, fieldname, 'read' or 'write', user, document?)`	Boolean check

Function	Purpose
`assertFieldPermission(...)`	Same arguments, throws `PermissionError`
`getVisibleFields(doctype, user)`	Field names the user may read
`getEditableFields(doctype, user)`	Field names the user may write
`filterDocumentByFieldPermissions(doctype, document, user)`	Returns a copy with only readable fields

10.3 Row permissions

`nodra/permissions/row-permission` restricts **which records** a user sees, using rules like "this user may only see customers where `territory` is in a given list".

- `hasRowPermission(context)` checks one document.
- `getRowPermissionFilter({ doctype, user, userPermissions })` returns a filter object such as `{ territory: ['in', ['North','East']] }`, or `{}` for System Manager.
- `applyRowPermissions(...)` filters an array of documents.

The filter format uses operators, but `orm.getList` only supports equality, so you must apply these filters yourself (for example with `applyRowPermissions` on the loaded list).

---

11. Submittable DocTypes, docstatus and workflow

- `docstatus` (0 Draft, 1 Submitted, 2 Cancelled) is a real column and is saved by both `insert` and `update`.
- `is_submittable` and the `submit`, `cancel`, `amend` permissions are accepted, but **nothing in the ORM performs a submit or a cancel**. You implement the flow yourself.
- The recommended way is the **workflow engine** (`packages/nodra/src/workflow`). Its states carry a `doc_status`, so a transition can move `docstatus` for you. It is a standalone state machine, not wired into the ORM, so you call it from your server function and then save.
- If you use a workflow, **declare the state field** (for example `workflow_state`, type `Data`) in the DocType, or it will never be saved.

```
const data = doc.getData();
await workflowExecutor.applyTransition('Leave Request', data, 'Approve',
user.email, roles);
doc.set('workflow_state', data.workflow_state);
doc.set('docstatus', data.docstatus);       // set BEFORE update
await app.orm.update(doc);
```

---

12. Migrations and changing a DocType

```
pnpm run migrate                        # root script, runs the app's migrate
command
pnpm --filter <app> cli migrate --verbose
pnpm --filter <app> cli migrate --doctype-dir=/path/to/doctypes
```

What migrate does for each DocType

1. Saves the definition into the metadata tables (`tab_doc_type` , `tab_doc_field` , `tab_doc_perm`).
2. If the table does not exist: `CREATE TABLE IF NOT EXISTS` plus indexes.
3. If it exists: `ALTER TABLE ... ADD COLUMN` for each **new** field.
4. Indexes created (with `IF NOT EXISTS`): `modified` , `owner` , `parent` and `parenttype` for child tables, every `Link` and `DynamicLink` field, and every field with `search_index` .

Migration history is recorded, but skipping unchanged DocTypes is currently switched off (`force = true`), so every run re-syncs everything.

Which folder is scanned. Migrate scans only `<cwd>/doctypes` unless you pass `--doctype-dir` . At runtime the app also loads the framework DocTypes (`packages/nodra/doctypes`). To create the core tables (`tab_user` , `tab_role` , `tab_user_role` ...), run migrate once with `--doctype-dir` pointing at the framework folder. Check this against your setup.

What a migration will and will not change

You change	Result
Add a new field	Column added. Safe

You change	Result
Add a `reqd` field to a table that already has rows	**Fails**, because `NOT NULL` without a default cannot be added to existing rows. Add it as optional, fill the data, then tighten it by hand
Rename a field	A new column is added. The old column and its data stay
Delete a field	The column stays in the database
Change a field type, `reqd`, `unique`, `default` or `max_length`	**Not applied** to an existing column. Alter it with SQL
Change `Select` options	Takes effect after restart (the validator reads the registry). The database is untouched
Remove a DocType JSON	The table is **not** dropped

Migrate never generates `DROP COLUMN` or `DROP TABLE`. Destructive changes are always manual.

Checklist after editing a DocType: save the JSON, run migrate, restart the server, then test an insert.

---

13. API reference

Import paths use the package exports of `nodra`: `nodra/core/*`, `nodra/orm/*`, `nodra/auth/*`, `nodra/database/*`, `nodra/permissions/*`, `nodra/api/*`, `nodra/utils/*`. The `doctypes/` folder and `workflow/` are not exported this way.

13.1 Parsing and loading (`nodra/core/doctype/schema`, `.../loader`)

Function	Returns	Notes
`parseDocType(raw)`	`DocTypeDefinition`	Validates a raw object. Throws `ValidationError`
`injectStandardFields(doctype)`	new `DocTypeDefinition`	Prepends standard fields (and child fields). Does not mutate the input

Function	Returns	Notes
`loadDocTypeFromFile(path)`	`Promise<DocTypeDefinition>`	Reads, parses and injects. `NotFoundError` if the file is missing, `ValidationError` if invalid
`loadDocTypesFromDirectory(dir)`	`Promise<DocTypeDefinition[]>`	Recursive. Invalid files are skipped with a warning
`resolveDocTypePath(basePaths, name)`	`string` or `undefined`	Finds `<snake_name>.json`

Constants and helpers (`.../field-types`): `FIELD_TYPES` , `FIELD_TYPE_PG_MAP` , `isFieldType` , `getPgType(type, maxLength?)` , `isDataField` , `isNumericField` , `isTextField` , `isDateField` , `isLinkField` . Also `STANDARD_FIELDS` and `CHILD_TABLE_FIELDS` from `schema` .

Registering a DocType in code (useful in tests):

```
const def = injectStandardFields(parseDocType({
  name: 'Note', module: 'Core',
  fields: [{ fieldname: 'title', fieldtype: 'Data', label: 'Title' }],
}));
registry.register(def);
```

13.2 Registry (`app.registry`, class `DocTypeRegistry`)

Method	Returns	Throws
`register(doctype)`	void	`DuplicateError` if the name exists
`get(name)`	`DocTypeDefinition`	`NotFoundError`
`has(name)`	boolean	none
`list()`	all definitions	none
`listByModule(module)`	definitions in that module	none
`getLinkedDocTypes(name)`	names of DocTypes with a `Link` field to `name`	none
`clear()`	void (for tests)	none

13.3 Document (`nodra/core/document/document`)

Constructor: `new Document(meta, data?)`. Standard keys in `data` become properties, the rest are stored as field values. The special key `_isNew: false` marks the document as already saved.

Member	Description
`doctype`, `name`, `owner`, `creation`, `modified`, `modified_by`, `docstatus`, `idx`	Standard properties
`get(field)` / `set(field, value)`	Read and write any field
`getData()`	Plain object of all values, including a `doctype` key
`isNew()`	True until saved or loaded
`hasChanged(field)`, `getPrevious(field)`, `getChangedFields()`, `markAsClean()`	Change tracking (section 8)
`getMeta()`	The DocType definition
`setIsNew(bool)`	Internal, used by the ORM

Member	Description
Hooks	`beforeValidate`, `validate`, `beforeInsert`, `beforeSave`, `afterSave`, `afterInsert`, `beforeDelete`, `afterDelete`, `onChange`

If you select only some columns with `getList({ fields })`, the other standard properties keep their defaults. Include `name` in `fields` when you need it.

13.4 ORM (`app.orm`, class `ORM`)

Method	Returns	Throws	Hooks and notes
`insert(doc)`	the same `Document`	`ValidationError`, database errors	Full insert lifecycle. Fills `name`, `owner`, `creation`, `modified`, `modified_by`
`getDoc(doctype, name)`	`Document`	`NotFoundError`	No hooks. Plain `Document`
`getList(doctype, options?)`	`Document[]`	database errors	No hooks. See options below
`update(doc)`	the same `Document`	`ValidationError`, database errors	Full update lifecycle. Writes every field that has a value, including `docstatus`
`deleteDoc(doctype, name)`	`void`	`NotFoundError`	`beforeDelete` and `afterDelete` run on a plain `Document`
`getCount(doctype, filters?)`	`number`	none	Equality filters
`exists(doctype, name)`	boolean	none	

Method	Returns	Throws	Hooks and notes
`getValue(doctype, name, field)`	the value, or `undefined` if the record is missing	none	One column only
`setValue(doctype, name, field, value)`	`void`	database errors	**Skips hooks and validation.** Only sets the value and `modified`

`ListOptions`

Option	Meaning
`filters`	`{ field: value }`, combined with AND, **equality only**
`fields`	Columns to select (default all)
`orderBy`	`'column'` or `'column desc'` (one column)
`limit`, `offset`	Paging. **There is no default limit**, so always pass one

Security: values are sent as query parameters, but identifiers (filter keys, `fields`, the `orderBy` column, and the `fieldname` in `getValue` and `setValue`) are placed into the SQL as written. Never pass names that come from user input; check them against `meta.fields` first.

Transactions. The ORM has no transaction support in its current form. The optional patch `orm-transactions.patch` adds `orm.transaction(fn)` and an optional `{ tx }` argument on every method.

13.5 Errors (`nodra/core/errors`)

Class	HTTP	Used for
`NodraError`	500	Base class (has `httpStatus`)
`ValidationError`	400	Bad data. Has `details: { field, message }[]`

Class	HTTP	Used for
`MandatoryError`	400	A required field is empty
`LinkValidationError`	400	A link target is missing (defined, but the validator does not throw it)
`AuthenticationError`	401	Not logged in
`PermissionError`	403	Not allowed
`NotFoundError`	404	Missing DocType or document
`DuplicateError`	409	Duplicate DocType registration
`InvalidStateError`	409	Invalid state change
`DatabaseError`	500	Query builder and database problems

Database constraint errors (for example a `unique` violation, PostgreSQL code `23505`) may reach you as a raw `pg` error. Catch them and check the code if you want a friendly message.

13.6 Naming helpers (`nodra/core/doctype/naming`)

Function	Example
`generateHash(length = 10)`	`'a3f9c01b2e'`
`generateFieldName(value)`	`' Ravi '` becomes `'Ravi'`. Rejects null and empty
`parseFormatPattern(pattern, counter)`	`('INV-{#####}', 42)` becomes `'INV-00042'`
`toSnakeCase(name)`	`'Sales Invoice'` becomes `'sales_invoice'`
`toTableName(name)`	`'Sales Invoice'` becomes `'tab_sales_invoice'`

13.7 Query builder (`nodra/database/query-builder`)

A small SQL builder used by the ORM: `new QueryBuilder(table).select(...)`, `.where(col, op, value)`, `.orderBy(col, dir)`, `.limit(n)`, `.offset(n)`, `.count()`, `.insert(data)`, `.update(data)`, `.delete()`, then `.build()` returns `{ sql, params }`. `UPDATE` and `DELETE` without a `WHERE` throw `DatabaseError`. Values are parameterized, names are not escaped.

13.8 Hook registry (not wired to the ORM)

`HookRegistryManager` (`nodra/hooks`) can register app hooks (`doc_events`, `scheduler_events`, `boot_session`, method overrides) and run them by priority. The ORM does not call it, and the `hooks` key in a DocType JSON is stored only. For now put your logic in controller hooks (section 8).

13.9 CLI

Command	What it does
`new:doctype "Name"`	Scaffolds the JSON and the controller in `doctypes/`
`new:server-fn "Name"`	Scaffolds server functions for a DocType
`migrate [--doctype-dir=PATH] [--verbose]`	Syncs DocTypes into tables and metadata
`console`	Opens a REPL with `pool`, `getDoc`, `getList`, `query`, `help()`. These helpers use raw SQL and return plain rows, not Documents (`getList` defaults to 20 rows)

---

14. Gotchas in one place

1. **Only the hash naming rule is applied.** For anything else, set `name` yourself.
2. **The `name` column has no PRIMARY KEY.** Check `exists` before inserting a custom name.
3. **`reqd` plus `default` still fails** unless you set the value in code.
4. **`Link` is just text.** No foreign key and no existence check.

5. **Child tables are manual.** Insert, read and delete child rows yourself. Never set a `Table` field on the parent.
6. **`getDoc` and `getList` return plain `Document`s.** Wrap them in your controller class to run hooks.
7. **`deleteDoc` skips your controller hooks.**
8. **`getList` has no default limit and only equality filters.**
9. **Identifiers are not escaped.** Never pass user-supplied field names.
10. **Currency values may come back as strings.** Use `Number()`.
11. **`Int` rejects numeric strings.** Convert form input first.
12. **JSON arrays need `JSON.stringify`.**
13. **Passwords are not hashed by the ORM.**
14. **Many properties are stored only:** `is_submittable`, `is_single`, `is_tree`, `is_virtual`, `title_field`, `search_fields`, `sort_field`, `hooks`.
15. **Migrate only adds columns.** It never renames, retypes or drops, and a `reqd` field on a table with data fails.
16. **No transactions** unless you apply the patch.
17. **ORM does not check permissions**, and role loading in the template is broken for child rows.
18. **Restart after any DocType change.**

---

15. Troubleshooting

Symptom	Likely cause	Fix
`DocType "X" not found`	JSON invalid and skipped, or server not restarted	Look for `Skipping <file>` in the log, fix, restart
`relation "tab_x" does not exist`	Migrate not run	Run migrate
`column "y" does not exist`	Field added but not migrated, or a typo	Run migrate, check `fieldname`
`"y" is a mandatory field` although it has a default	`reqd` plus `default`	Set it in code, or remove `reqd`

Symptom	Likely cause	Fix
`value must be an integer, got string`	Form data is text	Convert with `Number(...)`
Custom hooks never run	A plain `Document` was used	Use `new MyController(meta, data)`
Two rows share the same `name`	No PRIMARY KEY on `name`	Check `exists` first
SQL syntax error on a field	Reserved word as `fieldname`	Rename the field
Error when inserting a parent with child rows	You set the `Table` field on the parent	Remove it and insert children separately
`JSONB` array error	Array sent to a `JSON` field	`JSON.stringify` it
Everyone is treated as Guest	Roles are child rows and are not loaded	Query `User Role` by `parent`
Migrate fails adding a column	`reqd` field on a table with rows	Add as optional, backfill, then tighten

---

16. Cheat sheet

Creating a DocType, start to finish

1. `pnpm run new:doctype -- "Name"`
2. Fill in `fields` (no standard fields), `permissions`, and for child tables `is_child: true`
3. Add `Link` or `Table` fields for relations
4. `pnpm run migrate`, then restart
5. Write a controller if you need rules, and construct it yourself
6. Check permissions in your server function, then call the ORM

ORM at a glance

```
const app = await getNodra();
const meta = app.registry.get('X');

await app.orm.insert(new Document(meta, { ... }));
await app.orm.getDoc('X', name);
await app.orm.getList('X', { filters: { a: 1 }, orderBy: 'creation desc',
limit: 20 });
await app.orm.getCount('X', { a: 1 });
await app.orm.exists('X', name);
await app.orm.getValue('X', name, 'field');

doc.set('field', value);
await app.orm.update(doc);
await app.orm.setValue('X', name, 'field', value);   // no hooks, no
validation
await app.orm.deleteDoc('X', name);
```

Decision table

I want to…	Use
Point to another record	`Link` field with `options`
Hold many rows under a record	Child DocType plus `Table` field
Limit choices	`Select` with an `options` array
Make a value mandatory	`reqd: true` (and set it in code if it has a default)
Keep values unique	`unique: true`
Speed up lookups	`search_index: true`
Add a business rule	Controller `validate()` or `beforeSave()`
Control who can act	`permissions` plus `hasPermission` in your server function
Run an approval flow	The workflow engine, with a declared state field